Final Report: Personal Smart Locker Project

Patrick Rooney and Tyler Wood

Introduction

The Personal Smart Locker project for ECE 2804 focuses on the design and implementation of the electronics and control for a smart locker. At the heart of this project is an ATmega328P microcontroller. The primary objective is to use the ATmega328P to develop an embedded system capable of securely locking and unlocking a storage compartment using a password entered through a 12-key matrix keypad to move a servo motor. Our overall system is designed to enhance the experience and security of traditional package delivery/storage methods.

Beyond the core functionality of electronic locking and unlocking, this project implements additional features to enhance user experience, energy efficiency, security, and robustness. These enhancements include password change capability with non-volatile storage through the ATmega328P's EEPROM, password encryption, battery monitoring with user alerts, power-saving modes for extended battery life, and fault detection for motor blockage to prevent mechanical damage. The integration of user interface elements such as buzzers indicating the state of the system further improve the design by enhancing system usability and interactiveness.

The goals of this project extend beyond replicating commercial smart lockers; they emphasize optimization, control, and energy-efficient operations suitable for low-cost, standalone applications. This project provides a platform for applying principles of digital design, state machines, circuit design, and bare metal embedded programming, all while adhering to real-world constraints in reliability, power consumption, and user interaction. These considerations are fundamental in modern embedded systems engineering and are directly addressed in our implementation.

High Level Design

The locker project integrates several coordinated subsystems to create a functional embedded system that mimics the behavior of a smart locking device. Although the final prototype doesn't include a physical locking mechanism or casing, the hardware and software collectively emulate the essential operations of a secure electronic locker. A block diagram of the complete system is shown in *Figure 1*.

At a high level, the system is powered by a single 9 V battery that serves as the primary voltage source for all subsystems. Voltage across the subsystems of the locker is either supplied 9 V directly from the battery or provided from a 5 V regulator that the battery supplies to. The separation ensures that sensitive digital components such as the ATmega328P receive a stable, regulated low voltage while still allowing power-intensive components to operate without placing excessive load on the microcontroller.

The central processing element of the system is a standalone ATmega328P microcontroller, chosen for its balance of programmability, low power consumption, and availability of digital and analog I/O interfacing with all subsystems. The chip directly manages user input from a keypad, orchestrates system state transitions, and issues control signals to other components in different subsystems. Using the bare microcontroller rather than a full development board such as an Arduino Uno minimizes space usage, reduces system cost, and lowers power consumption, each considered valuable for battery-powered embedded devices.

The system operates around a structured finite state machine that defines the behavior of the locker. Upon startup, the system initializes the microcontroller and peripherals, asking the user to input a password to be saved in the system. Once the password has been registered, the system locks and only unlocks once the user inputs the valid passcode. The ATmega328P processes these inputs, compares the entered sequence to the stored password, and transitions to the appropriate state. Locked, unlocked, and relocking state transitions are indicated through different audio cues that utilize distinct frequency pitches in order to differentiate to the user which state the locker is in.

Several design decisions were made to ensure the system meets performance expectations for a battery-powered smart locker. The use of a standalone ATmga328p was chosen to reduce physical size, minimize power consumption, and avoid the cost of a full development board such as an Arduino Uno. A dedicated 5 V regulator was used to ensure reliable operation of digital components despite the natural voltage drop of a 9 V battery over its discharge cycle. The layout of the subsystems on the breadboards was laid out separately to improve modularity, allowing for easier testing and debugging as the design of the locker became more complex. These architectural decisions collectively support a system that is compact, energy efficient, and functionally representative of an actual smart locker.

Detailed Design and Verification

This section presents the schematic of the entire system of the Personal Smart Locker, as shown in *Figure 2*. Each subsystem is described with sufficient details of the individual components and connections pertaining to each subsystem that were discussed in the “High Level Design” section.

The keypad subsystem converts mechanical button presses into unique analog voltage signatures that the ATmega328P can decode using its Analog to Digital Converter (ADC) GPIOs. To minimize pin usage, the design implements a pull-up and pull-down resistor network, creating a voltage divider for each of the keypad’s three columns. Each column is connected to a dedicated ADC pin (A0 - A2), enabling three multiplexed voltage dividers, one per column.

This design creates a unique voltage divider for each key in each column when the 5V source is on. We calculated the voltage divider for each key in each column and created voltage ranges to account for impurities in circuit elements. We ensured that these voltage ranges were wide enough to accurately detect a key press for each voltage divider. We also made a small buffer zone between each voltage range in each column to ensure there was no risk of overlap in the ranges, causing a wrong key to be read. We chose these resistor values because their voltage dividers allowed me to create ranges that would not risk causing overlapping ranges for each column, as previously described. We also chose to use an individual GPIO for each column because it allowed me to have voltage ranges between the columns that overlap, without risking the wrong key press being detected.

After designing and building this circuit, we then mapped the voltage ranges to 0-1023 values to be read by an analog-to-digital converter (ADC) in the Arduino code we developed. These 0-1023 values became our minimum and maximum ADC values. The voltage divider calculations are shown below in *Figure 3,* and the 10-bit Arduino ranges are shown below in *Figure 4.*

We then used these 10-bit Arduino ranges and the circuit we built to develop code to control the keypad. We began by creating 4 global variables: one for the number of keys and one for each of the three columns of the keypad. We set column 1 to A0, column 2 to A1, and column 3 to A2 to configure the pins as wired in *Figure 2*. We then created a struct containing a character variable for the key

pressed, integer variables for maximum and minimum ADC values, and an integer variable for the column. Using this struct allowed me to make a matrix of vectors, which we called keyMap, where each vector contains all necessary values for the key presses (character value, minimum ADC, maximum ADC, and column). We put all of this together by creating a function called readKey(), which returns a constant character pointer. In readyKey(), we created three ADC integer variables and set each equal to an AnalogRead of the column value, with ADC1 being column 1, ADC2 being column 2, and ADC3 being column 3. We then looped through the number of keys, resetting a new ADC integer variable back to zero at the beginning of each iteration. Within this loop, we created three "if" statements that check the column element of the keyMap matrix, and set the new ADC to the value of the ADC corresponding to the column. We then put a final if statement in the main loop, which checks which range of minimum and maximum ADC values the ADC is in for the respective column. After determining the correct range, we then printed the character key value corresponding to that column and that range in the serial terminal, and the key became the return value of the function. The code for readKey() is shown in *Figure 5*. This implementation was able to accurately detect key presses, as shown by the output on the Arduino serial monitor in *Figure 6.*

Once we verified the keypad worked properly, we began to work on the servo motor. Without utilizing the external library functions built within Arduino, code was developed during the design process that involved directly accessing and manipulating certain register contents to create a functioning frequency divider that would divide the system clock frequency on the Arduino to output a 50 Hz frequency PWM signal that the servo motor requires to operate. This operation was built into a function called setup_50Hz() that would not return any specific data types. Research into the datasheet of the ATmega328P microcontroller chip suggested utilizing registers affiliated with Timer1 to accomplish this task. As compared to other timers built within the ATmega328P, this team chose to work with Timer1 due to its 16-bit register size. Compared to Timer0 or Timer2 on the microcontroller, Timer1 has a larger range of values it can count up to, which makes it easier to debug during the design process for this subsystem. It also allows for higher precision and stability when configuring prescaler values in order to output the correct frequency onto the servo motor. For reference, *Figure 7* in the appendix contains the block diagram of Timer1.

Initial understanding of the clock divider equation suggested having a division factor of 320,000 in order to divide 16 MHz down to 50 Hz. However, the behavior of timers in microcontrollers to produce certain PWM frequency signals involves configuring a correct prescaler value in order to accomplish this task. The equation used to find the correct prescaler that would be used within the implementation was

$$N = \text{Prescaler} * (f_{clk} / (\text{Prescaler} * f_{out}))$$
$$320{,}000 = \text{Prescaler} * (16\ \text{MHz} / (\text{Prescaler} * 50\ \text{Hz}))$$

Plugging in 8 for the prescaler value gave N to be equal to 320,000, suggesting that the prescaler value that would be used in the implementation would be 8. To improve the readability of the code, the prescaler was defined as a macro in the final implementation along with the division factor number, N.

Three registers, TCCR1A, TCCR1B, and TCNT1, were each reset prior to executing the code to run the motor in order to prevent any saved values in these registers from resulting in undefined behavior. Generating the correct PWM frequency would involve setting certain bits in each of the timer and

counter registers to 1. Defined within the datasheet for the ATmega328P, these bits would be referred to as WGM13, WGM11, and WGM12, COM1A1, and CS11. Predefined bitmasking macros within the Arduino IDE were used to set the correct bits high for registers TCCR1A and TCCR1B to set the correct waveform generation mode required to output a PWM signal of 50 Hz.

The initial setup for this function incorporated bitmasks onto registers TCCR1A and TCCR1B to set the correct bits in each register to 1 to set up the correct waveform generation mode. The initial function setup didn't accommodate for setting CS11 (bit 1) to 1 to allow Timer1's clock on the hardware chip to define the prescaler value that will be used as part of the frequency divider implementation. This was fixed in the final implementation and was defined as a macro.

Initial testing of the servo motor circuit generated two duty cycles to represent two different states that the angle of the motor turns to. The values used for the pulse widths of each duty cycle were set into OCR1, which would trigger the specific duty cycle event to Pin 9 on the Arduino in order to successfully switch the motor between two different states. Different values were tested with values ranging between 1000 to 4000 to be equated into OCR1A, causing the motor to turn successfully at different angles. *Figure 8* and *Figure 9* contain the PWM signals that were delivered to the VCC wire to the servo motor when OCR1A equals 1000 and OCR1A equals 4000.

The frequency measurements across Channel 1 on the AD2 for both states show a frequency that closely resembles 50 Hz with a period of 20 milliseconds. Measurements of the pulse widths for the locked and unlocked states differ, where the locked state has a pulse width of about 0.5 milliseconds and the unlocked state has a pulse width of 2 milliseconds. The two duty cycles successfully rotated the servo motor to its corresponding angular positions, effectively demonstrating the functionality of the subsystem in controlling the motor between locked and unlocked.

A state machine is also necessary to control the Personal Smart Locker. The implementation was structured around an FSM with five states: IDLE, LOCKED, INPUT_PASSWORD, VALIDATING_PASSWORD, and UNLOCKED (refer to *Figure 10* in the Appendix). This setup was configured to verify that the FSM could correctly traverse between states as it previously did while utilizing the Arduino. Numeric input was entered through the serial monitor to simulate user passcode entry, allowing validation of the transitions between states. *Figure 11* shows the output printed to the serial monitor when the FSM received an input that matched the provided password, while *Figure 12* shows the monitor printing when the password doesn't match what is expected.

During testing, a Boolean was used to trigger transitions between UNLOCKED and LOCKED states since the initial setup relied primarily on serial input for testing. This Boolean would be set to true once the current state was UNLOCKED. This would trigger an event in the code to then go from UNLOCKED to LOCKED after a certain number of seconds when the Boolean was true. *Figure 13* shows in the serial monitor how the transition between states successfully moves from UNLOCKED to LOCKED on the Boolean. Assuming the Boolean represents a switch or button to represent a lock button, the state behavior was able to transition as required.

We utilized a modified implementation of the FSM created above to test the password functionality of our system. We began by exploring different ways to save data to long-term memory on the Arduino. We decided that using the Arduino's EEPROM made the most sense, since EEPROM has memory that holds even when the board turns off. Using EEPROM ensures that the password cannot be hardcoded and is persistent even when disconnected from a power source.

Since we were not allowed to use external libraries, we had to create functions for the EEPROM directly in our code. We used the ATmega328P datasheet, since this is the microcontroller that the Arduino uses, to obtain functions for EEPROM_read() and EEPROM_write. EEPROM_read() reads from the EEPROM memory, and EEPROM_write() writes to the EEPROM memory. We utilized EEPROM_write() to create a function for setting or changing the password called changePassword(), which takes a character pointer as input. This function then loops from zero to four and in each loop does an EEPROM_write(), setting the inputted character value at the loop index to the address of the loop index in EEPROM. Once the loop is over, another EEPROM_write() is executed at the fourth index to null-terminate. It is important to null-terminate the fourth index because this prevents the stored password from being more than four characters. We then utilized EEPROM_read() to check if the input password is correct. This function is called readPassword() and takes a character pointer as input. Within this function, it loops from zero to four, and in each loop, it sets the input character array equal to the previously stored password values in EEPROM. After the loop, we null-terminate the fourth index of the input to prevent it from being over four characters long, since our password is only four characters long. We then created three global five-character arrays: currentPassword, enteredPassword, and newPassword. These arrays are all used at various times in our previously developed FSM, and work with changePassword() and readPassword() for creating new passwords and checking if passwords are correct for unlocking the system and changing passwords. The code for changePassword() and readPassword() is shown below in *Figure 14.*

Our implementation behaved exactly as expected. We tested this by using a modified version of the FSM that was previously created to input an original password for the locker (where a password has not been previously saved yet), save the password, and output the password read from EEPROM into the serial monitor. This FSM has states for setting the original password, checking the password, setting a new password, and being idle. This FSM also utilizes the readPassword() and changePassword() functions, the global password variables, if-else statements, strcmp functions, strcpy functions, and state switching. This output is shown below in *Figure 15.*

Once the basic subsystems were built, we then worked on getting our system to work off an ATmega328P chip rather than an Arduino and minimizing power consumption to optimize battery life. We began by developing a way to determine whether or not the servo motor was blocked, since a blocked motor could draw excess current, quickly draining the battery and could damage the physical components of our system. Our design had very simple circuitry and mostly relied on a helper function in our code to change states in our FSM if the motor was blocked. Since the motor changes states very quickly (less than 500ms), and it is not actively drawing current when it is not moving, our helper function is designed to read the voltage supplied to the servo motor 800ms after the motor initially starts moving to determine if it is blocked. If voltage is still being supplied 800ms after the motor starts moving, it indicates that the motor is blocked. It accomplishes this within one second to ensure that excess current is not drawn and the motor is not damaged.

We began our design by connecting a 1.8 ohm resistor to the servo motor. We chose a 1.8 ohm resistor because it is very small and the associated voltage drop does not cause any bugs in the servo motor functionality. We then connected pin A3 on the Arduino to the resistor to measure the voltage through the resistor by performing an analogRead() when the helper function is called. We then created our helper function, called motorBlocked(), which returns a Boolean value. If the function returns true, it

indicates that the motor is blocked, and if it returns false, it indicates that the motor is not blocked. We began by creating an integer variable for the servo threshold and set it equal to 75. We chose 75 because it is a value high enough not to be triggered by noise in the circuit, and it is low enough to detect if voltage is supplied through the 1.8 ohm resistor to the servo motor. We then configured an integer variable for the servo connected to pin A3 and performed the 800ms delay. After the 800ms delay, we created a new integer variable for the servo ADC and had it store the analogRead() value of pin A3, the GPIO we connected to the resistor. We then wrote an if statement to check if the value of the servo ADC is greater than the threshold value. If it is, this indicates that the motor is blocked, and the function returns true. If the servo ADC is not greater than the threshold, this indicates that the motor is not blocked, and the function returns false.

To ensure that the incorporated function is able to successfully detect blocking and nonblocking behavior of the servo motor, we tested both behaviors of the motor for our locker system. The first test consisted of checking if the motor would turn successfully when it wasn't blocked. *Figure 17* shows the results tested using the Serial Monitor. The behavior depicted in the monitor indicates that the motor successfully turned to its unlocked state.

Prior to validating that the program can run on a non-blocked motor, the motor was blocked during its next test to validate that it will not remain in its locked state when attempting to be unlocked with the correct password. *Figure 18* shows the output from the Serial Monitor. The monitor specifically indicates that the servo was blocked and proceeds to remain at its current state.

Once the motor blockage detection worked, we began to port our entire design so far onto a standalone ATmega328P chip. In order for the Atmega328P microcontroller to operate similarly to the microcontroller from an Arduino Uno, the chip requires a 5V source supplied to its power rail pins, the bootloader to be burned, and the program previously developed to be programmed directly to the standalone ATmega328P. Since our implementation leveraged the Serial Monitor on Arduino IDE for debugging and system behavior, the chip also needed a connection directly to Arduino IDE to route the program from the chip onto the Serial Monitor.

To configure the ATmega328P for standalone operation, a 9V battery source was regulated to a 5V supply using an MCP1702 voltage regulator. The regulator was accommodated for noise and ripple effects by using two 1 µF capacitors, each connected to opposing nodes of the regulator. These capacitors assisted in reducing input noise from the battery and decreasing output ripple effects from the regulator. *Figure 21* depicts a circuit diagram of this configuration. Once the regulator circuit was built, the voltmeter from the AD2 was connected to verify the 5V output from the regulator, as seen in *Figure 22*.

After validating, the output node supplying 5V from the regulator was connected to the necessary supply rails of the microcontroller, powering the device up. The necessary ground connections were made to the ground rails along with the Vcc supply rails of the ATmega328P, ensuring proper power delivery to operate the chip. *Figure 21* and *Figure 23* show the diagram connections and the associated pins with the Arduino Uno to the standalone ATmega328P.

The procedure to burn the bootloader required following the guidelines outlined in the Programming Standalone ATmega328P document provided within Canvas. Making the necessary connections from the Arduino Uno to the standalone ATmega328P, as depicted in *Figure 23,* would allow

the chip on the breadboard to operate just like our Arduino Uno. This would allow us to flash the locker program on the standalone chip without needing a direct connection to the Uno to make it happen.

Once the bootloader was burned and the previously developed program was flashed to the chip, the construction of the rest of the circuit to operate the locker in conjunction with the 9V battery and ATmega328P was made. The initial design of the locker done so far used the AD2 as the voltage source and ground to the resistor network to correctly map voltage ratings to individual keys on the 12-key matrix keypad. This source was replaced with the 5V output from the regulator and the 9V battery's negative terminal to ensure the voltage readings configured to each key were accurate to the initial voltage readings. Below is a table detailing the measured voltages across each key pressed on the keypad.

When the measured voltages in *Figure 24* were compared to the calculated voltage ratings in *Figure 3*, the measured results aligned closely, confirming that each key correctly sources its expected voltage level through the resistor network when pressed. Minor differences between the measured and theoretical values can be attributed to component tolerances in the regulator, battery, and the transmission wires across the circuit on the breadboard, which slightly affect the 5V reference across the pull-up and pull-down resistor network.

To fully validate that the circuit is able to correctly register each key press and successfully change states, the locker would require a visual representation of keypress behavior. Our initial idea was to utilize LEDs in a matrix-style configuration, correlating each LED to each key. However, this would have proven to be ineffective because the matrix configuration would set certain LEDs correlating to other keys to light up, along with the one that is supposed to light up. Also, due to the internal voltage drop across each LED (~0.7V), an LED correlating to a certain key may end up not lighting up. This would make it difficult to visualize which keys are being registered in the locker program. It would be inefficient as well because each LED would need to utilize additional space onto the breadboard that the locker circuit is being built on, increasing the complexity of the circuit when it can be optimized for simplicity. The alternative solution was to utilize the Serial Monitor within Arduino IDE by routing the Serial Print messages from the standalone chip to the IDE via an Arduino Uno connection.

By connecting the RX and TX pins from the ATmega328P to the adjacent pins on the Arduino Uno, messages programmed on the standalone ATmega328P could now be interpreted from the Serial Monitor. To help further in debugging the key output to the Serial Monitor, a connection to the analog reference pin from the chip to the Arduino Uno was made to provide an external reference voltage for the ADCs to the analog pins to correctly map analog values to their respective keys on the keypad within the locker's program. *Figure 25, Figure 26,* and *Figure 27* show the results from the Serial Monitor testing that the standalone chip outputs the expected state behavior for the locker.

Another common way to improve battery life is using the built-in sleep modes on the ATmega328P. We successfully implemented a sleep mode on our microcontroller to minimize idle current while ensuring our system can exit sleep mode to provide all necessary functionality for the Personal Smart Locker. We chose to use power-down mode as our sleep mode. This sleep mode is the strongest sleep mode offered on the ATmega328P, providing the lowest power consumption by using only one micro amp at three volts. It can also wake up on pin change and timer interrupts, which we utilized in our implementation to ensure the system is awake when a user is interacting with the keypad and when the motor is locking or unlocking our system.

We began our implementation by creating a general sleep function to put our system into power-down mode whenever it is idle. We initially programmed this function using the sleep.h library, but ended up utilizing the ATmega328P datasheet to program directly to the registers. We created the function by setting the sleep mode to power-down mode, enabling sleep mode, setting the CPU to sleep, then waking the system up again by disabling the sleep mode. We also included print statements indicating when the system goes to sleep and when it wakes up for testing, as shown in *Figure 28*.

With our system able to enter a sleep mode, we had to develop interrupts to ensure the system could exit sleep mode and remain fully functional. We began this by developing pin change interrupts. In setup(), we enabled the pin change interrupt for A0-A5 by setting bit 1 of the pinchange interrupt control register to 1, by writing the line "PCICR |= (1<<PCIE1)". Within the A0-A5 group, we then enabled pin change interrupts for specifically A0, A1, and A2 by writing "PCMSK1 |= (1<<PCINT8) | (1<<PCINT9) | (1<<PCINT10)". After configuring the interrupts for A0-A2, we created an ISR function using a global volatile Boolean variable called keyInterrupt, which is initially set to false. The purpose of the ISR is to wake the system from sleep mode. The ISR takes an argument of PCINT1_vect to read the pin change interrupts we configured, then sets the keyInterrupt flag to true, as shown in *Figure 29*.

We then worked to develop timer interrupts, using Timer2, to ensure the system is not in sleep mode while the motor is running. We decided to have the system wake up for five seconds when the motor is locking or unlocking. Waking for five seconds allows the motor to have enough time to fully change from the locked to unlocked position and vice versa. We began our implementation by creating three global variables: a global Boolean called motorRunning for main loop logic, a global Boolean called motorTimerExpired for the interrupt flag, and a global int called motorTimerCount to keep track of how many times the timer runs in CTC mode. We then created a function to start the motor timer. In this function, we set register A, register B, and the counter for Timer2 to 0. We then set the CTC compare value to 2ms by setting 0CR2A to 31. We found the value 31 by using the formula: 0CR2A = (((Fcpu*desired time)/prescaler)-1). We chose to use a prescaler of 1024 and a desired time of 2ms. Since the system clock frequency is 16,000,000, this simplifies to 0CR2A = (((16,000,000 *0.002)/1024)-1), and simplifies further to 0CR2A = 31. We then set the timer to CTC mode, set the prescaler to 1024, and enabled the interrupt to trigger when Timer2 matches the calculated value for 0CR2A, as seen in *Figure 30*.

Next, we created an ISR function for the motor timer. This function counts up each time the timer hits the desired value of 500ms using the global motorTimerCount variable I created earlier and the desired 2ms count we created earlier. Once the count motor count hits ten (5 seconds have passed since the timer started), the timer expired interrupt flag is set to true, the motor running flag is set to false, and the Timer2 Match A interrupt is disabled. Disabling the Timer2 Match A interrupt means the ISR will not be triggered again until a new interrupt from the motor triggers it. This function can be seen in *Figure 31.* When the motor is running, we reset all flags and counters for our timer interrupts: motorRunning is set to true, motorTimerExpired is set to false, and motorTimerCount is set to zero. When the motor stops running, we also reset the motorRunning flag back to false.

We implemented the logic to ensure proper power-down mode for both the keypad and servo motor into our main loop by first checking if the key interrupt flag and the motor running flag are both set to false. If they are, it indicates to the system that no key is currently being pressed and the motor is

not running. This means it is ‘safe’ for the system to enter sleep mode, so when both of these flags are false, the system then enters sleep mode. This implementation can be seen in *Figure 32.*

We tested our results using the print statements in our enterSleep() function. As you can see in *Figure 33*, the system functions properly with power-down mode incorporated for each key press. To test our timer interrupt, I edited our FSM to print a statement to indicate when the motor is running. As you can see in *Figure 34,* the system properly goes to sleep, wakes up on a timer interrupt when the motor is running, and returns to sleep after five seconds.

Now that the Personal Smart Locker is able to perform all basic functionality independent of an Arduino, we began to develop ways to improve the user interface and security of the overall system. The first way we improved the user interface was by creating a detection system to notify the user when the battery is low. The hardware for our design consists of two resistors in series, connected to the 9V battery, to create a ½ voltage divider and perform an analog read between the two resistors. The voltage at the midpoint of the divider is half of the battery voltage. So if the battery is at 9V, the analog pin will see 4.5V. This makes it safe to measure the battery level with a 5V ADC. These resistors are essential because, without them, we would not be able to perform an analog read to measure the voltage through the battery. We chose two 10k ohm resistors for this design because they create the ½ voltage divider we need. Once the battery voltage drops below 6.5V, we will blink an LED in our circuit to indicate that the battery is low. We chose 6.5V as our threshold because our battery is connected to a 5V voltage regulator to power our circuit, and once the voltage supplied to the voltage regulator drops below 6V, it stops working. We determined that having the 0.5V buffer is sufficient to give the user time to replace the battery before it drops below 6V.

We began working on this design by configuring our software to allow for an LED to blink on the circuit. We first created a global constant integer variable called ledPin and set it equal to 13. We did this because we are using the digital pin 13 on our ATmega328P for the LED functionality. We then added the line “pinMode(ledPin, OUTPUT)” into our setup() to configure this pin as an LED output.

Once the LED configuration was finished, we began working on our helper function for determining low battery in our system. The first thing we did was create our low battery threshold. Since the voltage divider scales 9V to 4.5V, a 6.5V battery maps to 3.25V at the ADC pin. The corresponding ADC threshold is calculated as: threshold=((3.25V/5V)*1023)=665. We divided by 5V instead of 4.5V because the ATmega328P analog read is performed on a 5V range and multiplied by 1023 to convert this value to a 1023-bit range. We then defined a function called lowBattery(), which returns a Boolean value. Within this function, we created an integer variable called lowBatteryThreshold and set it equal to 665 (the value we previously calculated), an integer variable called lowBatteryGPIO and set it equal to A5 (since we are using pin A5 for the analog read), and a final integer variable called lowBatteryADC and set it equal to an analog read of the lowBatteryGPIO. We then created an if statement that checks if lowBatteryGPIO is less than lowBatteryThreshold. If it is, this indicates the system's battery is low, and the function returns true. If it is false, it indicates that the system’s battery is not low, and the function returns false. This function can be seen in *Figure 35.*

We then implemented this function in our main loop logic. To ensure that this function works properly with our already implemented sleep mode, we added a line to our conditional statement that enters sleep mode. We did so by changing “if(!keyInterrupt&&!motorRunning)” to “if(!keyInterrupt&&!motorRunning&&!lowBattery())”, so if the battery is low, the system will not enter

sleep and the LED will still blink to indicate the low battery to the user. This logic can be seen in *Figure 36.*

We then added logic to our main loop to make our LED blink when the power is low. I wrote a statement to check if lowBattery() returns true. If it does, then we perform a digital write on ledPin to force it high (turns the LED on), delay 1000ms, perform another digital write on ledPin to force it low (turns the LED off), and delay another 1000ms. This logic can be seen in *Figure 37.*

We tested this system by first connecting the power rail of our breadboard to the 9V battery and configuring the circuit as described above, with the resistors connected directly to the power rail (9V source). As you can see in *Figure 38,* the LED does not blink to indicate low battery when connected to the 9V source, as expected. We then tested this system by connecting the power rail of our breadboard to 5V from the Arduino. As you can see in *Figure 39,* the LED blinks indicating low battery when connected to the 5V source because it is below the 6.5V low battery threshold, as expected.

We then further improved upon the user interface by adding a piezoelectric buzzer to indicate the state of the system to the user. Distinct audio cues are generated by the buzzer to indicate specific state transitions within the locker's finite state machine, providing the user with immediate auditory feedback corresponding to the system's current status. In particular, three distinct audible frequencies were implemented to distinguish the locker from being locked by the user on changing the password, the locker being unlocked on a correct password input, and the locker remaining locked on an incorrect password.

To minimize current draw across the buzzer, a bias resistor was added in series with the component. When selecting an appropriate resistor value for the bias resistor, the internal DC resistance of the buzzer had to be taken into consideration, which turned out to be 15.3 Ohms. The buzzer is driven by a digital output pin of the ATmega328P, which supplies 5 V and can safely provide 20 mA, which is about half of the 40 mA current limit of an Arduino I/O pin.

Applying Ohm's Law, the equivalent resistance required to limit the current to 20 mA is 250 Ohms. Subtracting the buzzer's internal resistance yields a calculated bias resistor value of 234.7 Ohms. The closest physical resistor value available for use is the 270 Ohms resistor, which also reduces the actual current draw to approximately 17.5 mA.

To implement the audio queue frequencies used in order to implement the buzzer feature, a function was created as a helper to replicate the behavior of the tone() function in Arduino IDE, because the timer resource used by tone() in the program was allocated for a different feature in the locker system. The helper function successfully generated a square wave of specified frequency and duration on a designated pin. The function, illustrated in *Figure 40*, forms the basis for implementing the three-state audio queue functions in *Figure 41*. As shown in *Figure 41*, each function uses distinct frequency patterns to ensure the states are audibly distinguishable. These functions are invoked within the main loop of the program as part of the locker's FSM architecture.

To verify that the audio queues trigger correctly during normal operation, the buzzer was tested alongside the full locker system. Pin 11 was selected as the audio output pin due to it not being utilized yet and the fact that it supports the square-wave generation required by the helper function. *Figure 42* links to the video file demonstrating the *correctPasswordAudioQueue()* function playing when the user inputs a valid password. Additional test recordings are provided in the Appendix to demonstrate the

additional audio functions that play. The observed behavior in the video files confirms that the audio cues play without interfering with system execution.

We also successfully implemented a security add-on for securely saving the password, so if anyone gained access to the physical system or memory, they would not be able to determine the password by encrypting the password data in EEPROM. Our choice for encryption was XOR encryption because it is one we have previously worked with, it is easy to work with, and it is secure enough to prevent the vast majority of malicious individuals.

We first created a define statement for our XOR encryption key. We made our key 0x5A and added "#define XOR_KEY 0x5a" to the top of our code file. We then edited the function we previously created for saving the password, changePassword(). We kept the function mostly the same; however, now when the user inputs data via pressing a key, we perform an XOR with the entered key and the encryption key before performing the EEPROM_write() to save the data to memory on the EEPROM. We accomplished this by adding a new char variable for the encrypted character within the loop (the loop iterates four times for the four-digit length of the password as described in previous reports), and performing an EEPROM_write() with the new encrypted character within the loop. This can be seen in *Figure 45.*

Now that the data is encrypted, we had to decrypt the data so it could be read properly within the FSM. To accomplish this, we edited the readPassword() function we had previously created. This function remained almost entirely the same as we had in our previous implementations. All that we changed was that we performed an XOR with the encryption key and the EEPROM_read() of the data in the EEPROM. This can be seen in *Figure 46.* This works for decrypting the data because XOR is reversible. When an XOR is performed twice with data and the same XOR value, the data returns to its original values, so A^B^B = A. As you can see in *Figure 47,* our implementation worked exactly as expected. The encrypted password in EEPROM is different from the password entered by the user, and all password functionality still works.

We also wanted to ensure the system we designed would be able to run as long as possible on a single battery before being replaced, and determine more ways to improve the battery life. To evaluate the efficiency of the locker system, the current draw of the system was measured under two primary operating conditions: low-power mode and active operation during a complete lock/unlock cycle. These measurements would allow us to determine the battery life of the 9V battery and also allow us to make design improvements to reduce current consumption.

In lower power mode, the current measured across the battery was 0.56 mA. In most typical 9V alkaline batteries, the mAh ranges from 500 - 600 mAh, depending on the brand. For best engineering practice, we assumed that the 9V battery was given 550 mAh to avoid overestimating the battery life. Assuming the system remains in low power for the entire life duration of the battery, the total hours of sleep-only operation would be calculated as:

$$550 \text{ mAh} / 0.56 \text{ mA} = 982.1 \text{ hours} = 40.9 \text{ days}$$

In the active case, where the system checks the password and turns the motor, the current when the servo is in the locked state is 4.8 mA and 5.3 mA in the unlocked state. The average of both cycles was calculated with the following equation:

$$I = (4.8 \text{ mA} + 5.3 \text{ mA}) / 2 = 5.05 \text{ mA}$$

The active duration for when the locker switches states, when playing the audio cues from the buzzer, and the servo motor moving can be used to calculate how many lock/unlock cycles can be utilized from the battery. When unlocking the system, the total audio time for the audio function in this state lasts for 1 second, and once the audio finishes playing, the servo moves, which is roughly 150 milliseconds since it is instantaneous, assuming the motor isn't blocked. With this, the calculated total time for unlocking the locker would be 1.15 seconds. When relocking, the total relock time takes about 0.35 seconds due to the time of the audio function and the servo motor movement. From the two values calculated for each different cycle, the total active time was calculated as:

$$T = 1.15\ s + 0.35\ s = 1.50\ s$$

From the calculation above, the locker system operates at about 1.5 seconds per full lock-unlock cycle. To compute the mAh used for each lock/ unlock event, the average active current was multiplied by the active time in hours, which is calculated as:

$$t(hours) = (1.50\ s / 3600\ s) = 0.0004167\ h$$

$$mAh\ per\ cycle = 5.05\ mA * 0.0004167\ h = 0.002104\ mAh$$

Using the 550 mAh of the 9V battery, the number of active cycles the locker can perform was calculated as:

$$550\ mAh / 0.002104 = 261{,}406\ cycles$$

Ignoring sleep consumption, the locker system performs about 261,406 lock/unlock cycles on a full 9V battery source. Improvements were made to the locker to improve the battery life of our system.

To improve the battery life of our system, we developed an implementation to eliminate constant current flow through the voltage divider used for battery monitoring. To accomplish this, we used a 2N7000 NMOS transistor that acts as a switch to control when the bottom resistor in the voltage divider is connected to ground.

In our circuit, the drain of the 2N7000 is connected to the end of the second resistor, the gate is connected to digital pin 12 of the ATmega328P, and the source is connected to ground. Controlling the gate through digital inputs/outputs allows us to control the current path through the voltage divider network as needed. When the digital write forces the 2N7000 high, it connects to ground, forcing the circuit to be closed, allowing current to run through the voltage divider network. When the digital write forces the 2N7000 low, it is disconnected from ground, forcing the circuit to be open, and does not allow for current to run through the voltage divider network. Utilizing this allows us to turn the 2N7000 on only when we need to check the voltage in the battery, and allows the 2N7000 to be off at all other times. This greatly reduces the current used by our battery monitoring system, which we had previously developed.

After we constructed the circuit as described, we developed the code needed to control our implementation. We began by creating a global constant integer called nmosPin and set it equal to 12, since we are using digital pin 12 to control our 2N7000. We then configured digital pin 12 in setup() by adding the lines pinMode(nmosPin, OUTPUT), and digitalWrite(nmosPin, LOW). This allows us to use nmosPin as needed in our design, and ensures that the two-resistor network is initially an open circuit when our program first runs.

Once everything in our code was properly initialized, we modified our lowBattery() function to produce proper switching behaviors for our 2N7000. Right before we initialized the lowBatteryADC and read the battery voltage, as described previously in the report, we performed a digital write to pin 12,

forcing the nmos pin high, and then added a 10-microsecond delay. This forces the 2N7000 to be high, closing the circuit for our two-resistor network and allowing current to flow. The short delay also allows for the voltage to be stabilized before reading it in our system. This then allows us to perform the analog read for the lowBatteryADC, check if the battery is low, and blink the LED to indicate low battery if needed. As soon as the battery voltage is read and stored in lowBatteryADC, we added another line performing a digital write to pin 12, forcing the nmos pin to be low again. This opens the circuit for our voltage divider network and prevents current from flowing, improving the battery life of our system. The updated lowBattery() function with the 2N7000 can be seen in *Figure 48.* We tested the functionality of our implementation the same way as we previously tested in our report, and the outputs are the exact same. The LED is off when the system does not have a low battery, as seen in *Figure 38*, and the LED blinks when the system's battery is low, as seen in *Figure 39.*

Validation of Overall Project

To verify proper operation of the complete locker system, a full demonstration was recorded and is included in *Figure 49*. The video captures a full interaction sequence with no manual intervention beyond normal keypad use. The recording confirms that the keypad correctly detects user input, the system transitions through the appropriate FSM states, password validation functions as intended, and both correct and incorrect entries trigger their respective audio cues. Additionally, the servo motor responds with the proper locked and unlocked motions driven by the Timer1-generated PWM, and the system remains stable and responsive during repeated lock-unlock cycles when a power supply is enabled.

At the start of the demonstration, the system prompts the user to create an initial password. The first four keypad inputs are detected and stored when the user presses '#', after which the password is saved in encrypted form to EEPROM. This also highlights a minor shortcoming: only four-digit passwords are supported, which may be restrictive for users expecting longer numeric codes. Throughout the demonstration, the Serial Monitor output further validates that each keypad press falls within the correct ADC voltage range defined earlier in the Detailed Design and Verification section.

Upon saving the password, the video demonstrates that the locker transitions correctly to the appropriate states depending on whether the password matches what is saved or not. The video shows the transition from locked to unlocked with the affiliated audio cue and servo motor transition when going to unlocked. The text prompting when the servo motor starts was used for debugging purposes, affiliated with detecting blocking behavior. In the unlocked state, the prompt asks the user to input a new password to be saved. Once the '#' key is hit, the servo transitions back to the locked state and triggers the locked audio cue to play. The last few seconds of the video detail the locker behavior when an incorrect password is inputted. As shown, the servo motor doesn't transition to the unlocked state and stays where it's at, ensuring that the pulse widths measurements stay in the closed state associated with the locked state. The audio cue affiliated with an incorrect password check also plays as expected.

Conclusion

The development of the Personal Smart Locker provided valuable experience and insight into the complexities and constraints in embedded system design. Through an iterative design process, we implemented a system that, in addition to meeting initial functional requirements, also addressed

real-world constraints such as power efficiency, robustness, physical component protection, and user interface development.

One of the many valuable lessons we learned from this project was the importance of an iterative engineering design process. Throughout this project, we encountered numerous challenges that could be approached in many different ways and could not be solved with a single, unchanged plan. Each subsystem, especially the keypad interface, power efficiency considerations, and motor control, required several cycles of planning, implementation, and debugging. This process taught us how to apply an engineering mindset by breaking down complex problems, isolating issues, and designing solutions based on real-world constraints. By the end of this project, constant use of this skill improved our ability to adapt and think critically, specifically in engineering applications.

If given the opportunity to do this project again, we would probably consider more how each step we made toward completing the project affected the final goals of the project to allow us to work more efficiently. For example, in our second milestone, we had to develop a way to save the password to long-term memory on the ATmega328P, and in our fourth and final milestone, we had to develop a way to safely store the password so that if someone gained access to the physical system, they would not be able to determine the password. As mentioned in our “Detailed Design and Verification” section, we chose to use XOR encryption when saving the password, which only required us to add one print statement and perform an XOR with our encryption key when storing and reading the password. This encryption could have easily been implemented when we initially developed our implementation for storing the password in long-term memory, and doing so would have allowed us to work more efficiently towards our final goals for this project. If we had more time to work on this project, we would also probably add an LED display to improve the User Interface by making our system more visually interactive.

Overall, this project required us to utilize prior course knowledge in digital logic, embedded programming, and circuit design, while also incorporating general engineering skills of debugging, optimization, and system integration to meet realistic design constraints.

Authorship Page

- 11/28/25 - Patrick created a draft passage for the Introduction section
- 11/30/25 - Tyler created a draft for the high level design diagram and elaboration on the diagram in the High Level Design section.
- 12/01/25 - Patrick compiled and revised all information from the Milestone Reports and created the final circuit schematic to complete the Detailed Design and Verification section of the report. Patrick also placed all images in the appendix to maximize space for description given the page limit on the report.
- 12/02/25 - Patrick completed the conclusion section of the report.
- 12/02/25 - Tyler revised and completed the High Level Design section of the report.
- 12/02/25 Patrick proofread and corrected grammar and writing quality in all sections besides Validation.
- 12/02/25 - Tyler completed the Validation of the Overall Project section and discussed aspects of the design verification video
- 12/02/25 - Tyler included the overall code ran on the locker within the verification video

Appendix

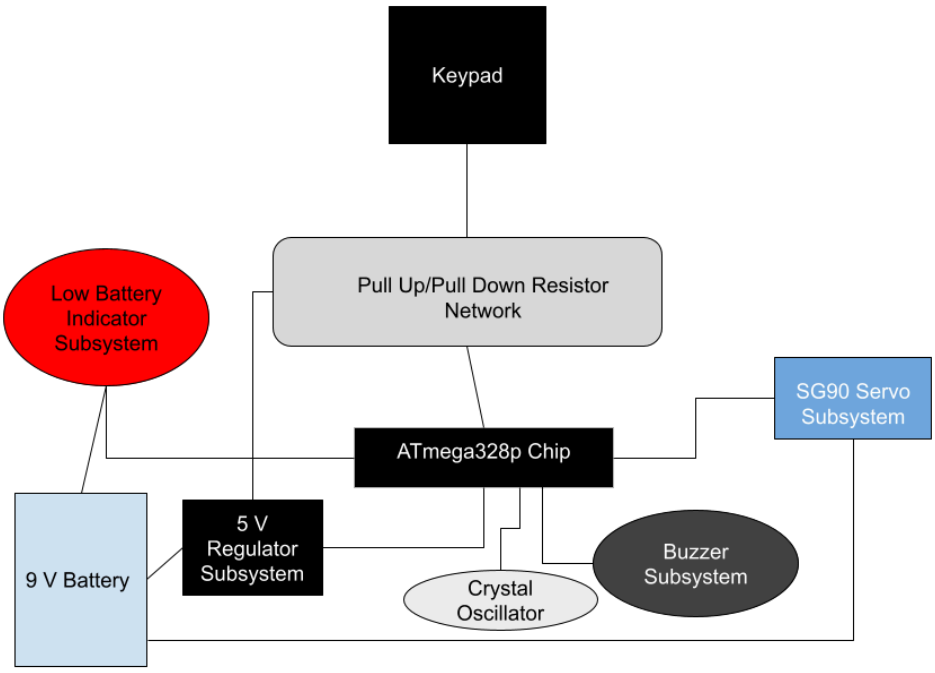

Figure 1: High Level Locker Design

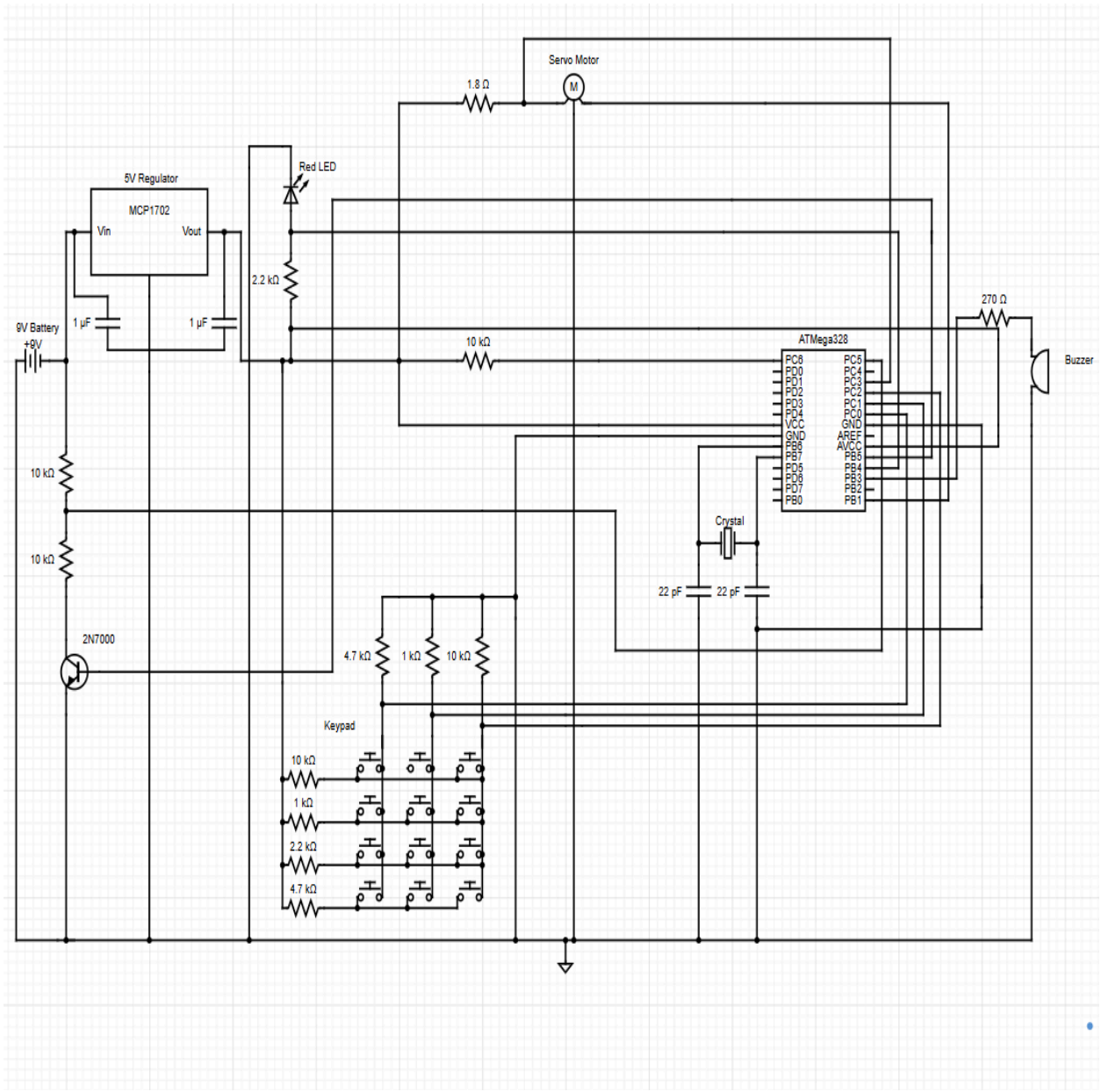

Figure 2: Schematic of the completed circuit

	Column 1	Column 2	Column 3
Row 1	5(4.7k/14.7k) = **1.60V**	5(1k/11k) = **0.45V**	5(10k/20k) = **2.5V**
Row 2	5(4.7k/5.7k) = **4.12V**	5(1k/2k) = **2.50V**	5(10k/11k) = **4.55V**
Row 3	5(4.7k/6.9k) = **3.40V**	5(1k/3.2k) = **1.56V**	5(10k/12.2k) = **4.1V**
Row 4	5(4.7k/9,4k) = **2.50V**	5(1k/5.7k) = **0.88V**	5(10k/14.7k) = **3.4V**

Figure 3: Voltage Divider Calculations For Keys

	Column 1	Column 2	Column 3
Row 1	225 ↔ 389 => **1**	61 ↔ 112 => **2**	430 ↔ 593 => **3**
Row 2	820 ↔ 1023 => **4**	430 ↔ 593 => **5**	900 ↔ 1023 => **6**
Row 3	634 ↔ 798 => **7**	265 ↔ 389 => **8**	788 ↔ 870 => **9**
Row 4	430 ↔ 593 => *****	143 ↔ 225 => **0**	634 ↔ 747 => **#**

Figure 4: Voltage Ranges Mapped To 10-Bit Arduino Ranges

```
const char* readKey(){
  int adc1 = analogRead(col1);
  int adc2 = analogRead(col2);
  int adc3 = analogRead(col3);

  for(int i=0; i<numKeys; i++){
    int adc = 0;

    if(keyMap[i].col == 1){
      adc = adc1;
    }
    else if(keyMap[i].col ==2){
      adc = adc2;
    }
    else if (keyMap[i].col ==3){
      adc = adc3;
    }

    if ((adc>=keyMap[i].min)&&(adc<=keyMap[i].max)){
      Serial.println(keyMap[i].key);
      delay(300);
      return keyMap[i].key;
    }
  }
  return NULL;
}
```

Figure 5: readKey() Function Code In Arduino

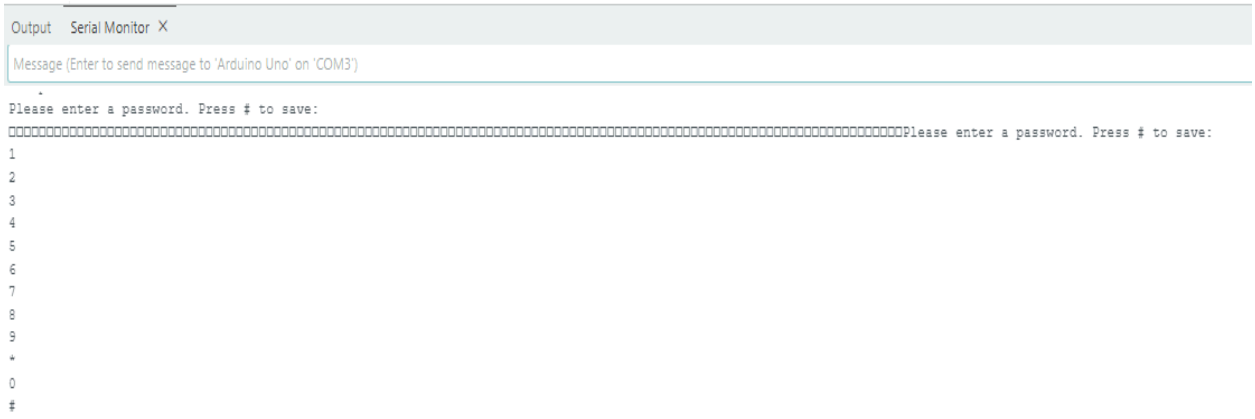

Figure 6: Serial Monitor Output For Keypad Testing

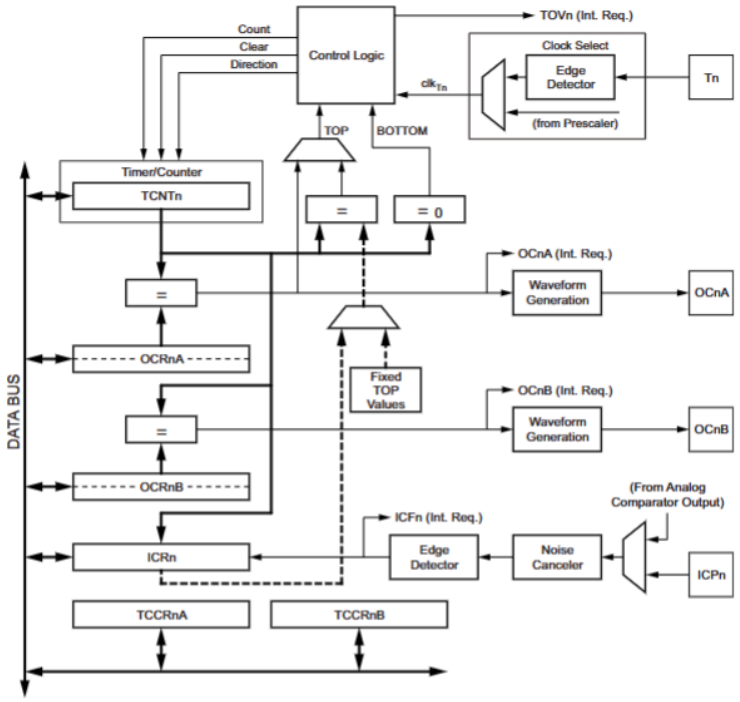

Figure 7: Block Diagram of Timer/Counter1 from ATMega328P Datasheet

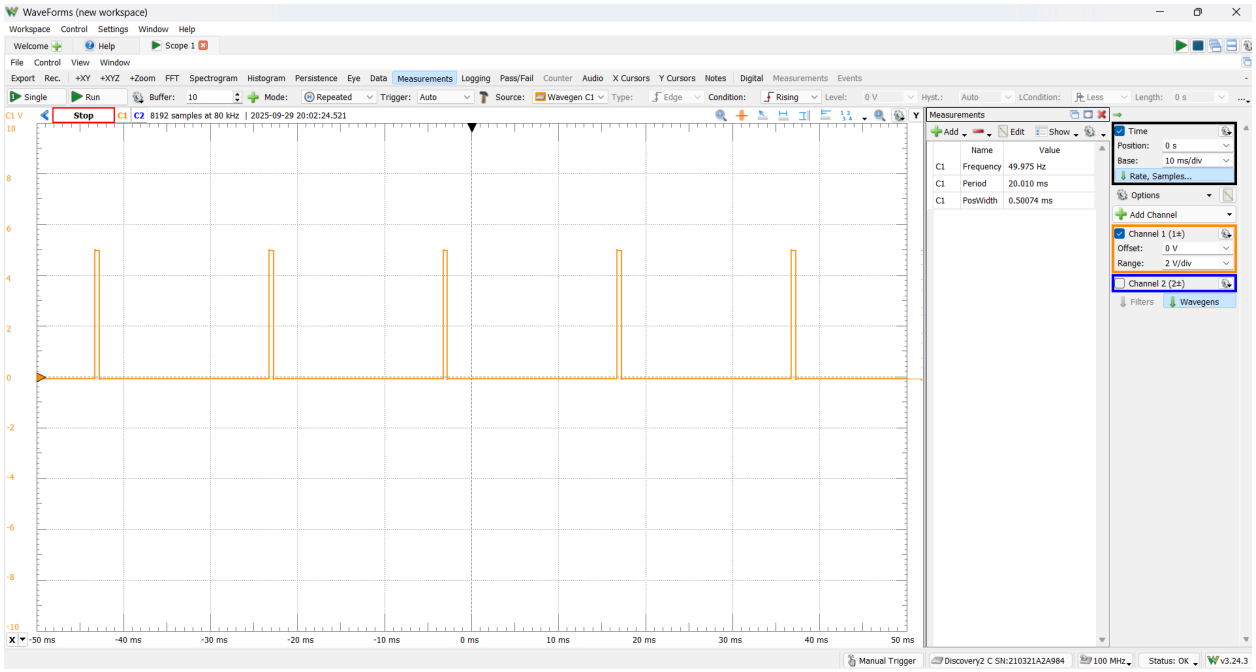

Figure 8: PWM 50 Hz Output Signal on Pin 9 for Locked State

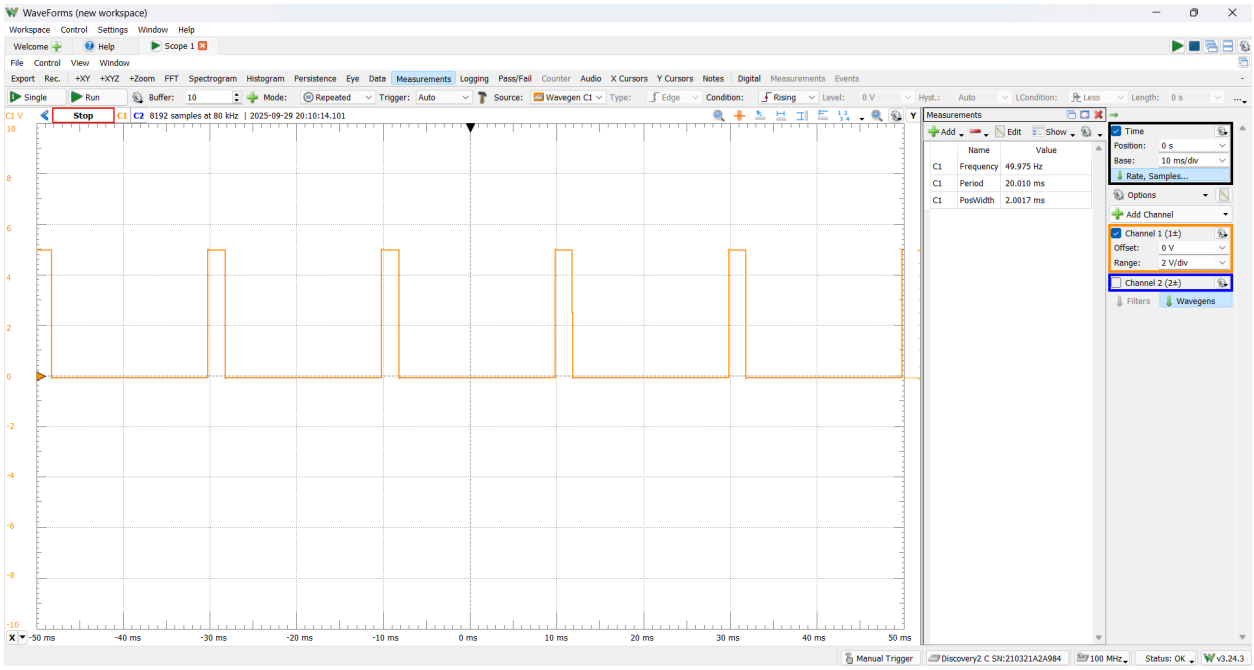

Figure 9: PWM 50 Hz Output Signal on Pin 9 for Unlocked State

```
enum State { IDLE, LOCKED, INPUT_PASSWORD, VALIDATING_PASSWORD, UNLOCKED, NUM_STATES };
State state = IDLE;
String savedPasscode = "3256";
String enteredPasscode = "";
bool lockButtonPressed = false;

void lockerStateChange() {
  char input = '\0';
  if (Serial.available() > 0)
  {
    input = Serial.read();
  }
  switch (state) {
    case IDLE:
      Serial.println("Idle -> Locked");
      state = LOCKED;
      break;

    case LOCKED:
      Serial.println("Locked. Waiting for input:");
      if (input != '\0') {
        enteredPasscode = "";
        enteredPasscode += input;
        Serial.println(input);
        state = INPUT_PASSWORD;
      }
      break;

    case INPUT_PASSWORD:
      if (input != '\0') {
        enteredPasscode += input;
        Serial.println(input);
      }
      if (enteredPasscode.length() == savedPasscode.length()) {
        state = VALIDATING_PASSWORD;
      }
      break;

    case VALIDATING_PASSWORD:
      if (enteredPasscode == savedPasscode) {
        Serial.println("Password correct!");
        state = UNLOCKED;
      } else {
        Serial.println("Passcode incorrect!");
        state = LOCKED;
      }
      enteredPasscode = "";
      break;

    case UNLOCKED:
      Serial.println("Unlocked");
      if (lockButtonPressed) {
        Serial.println("Lock button has been pressed");
        lockButtonPressed = false;
        state = LOCKED;
      }
      break;
  }
}
```

Figure 10: Password Behavior FSM Function

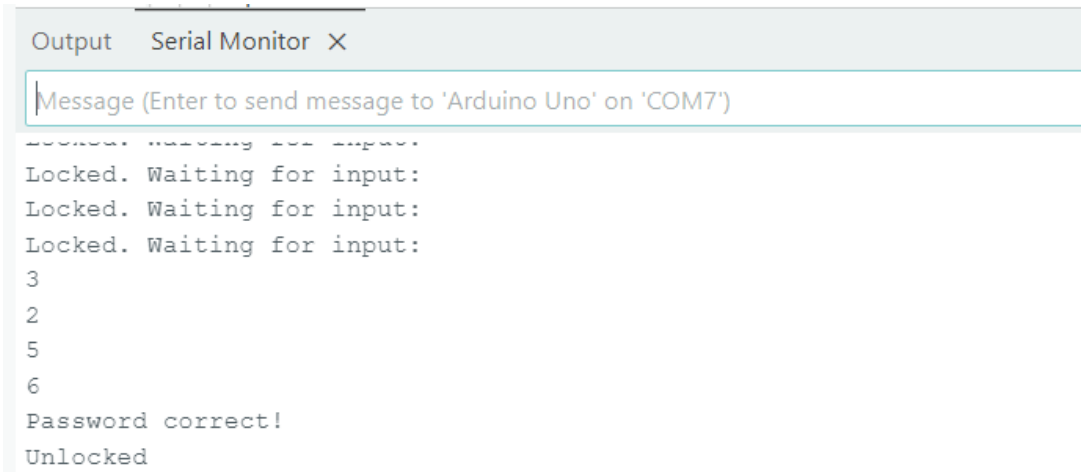

Figure 11: Serial Monitor Output in FSM Implementation on Correct Password

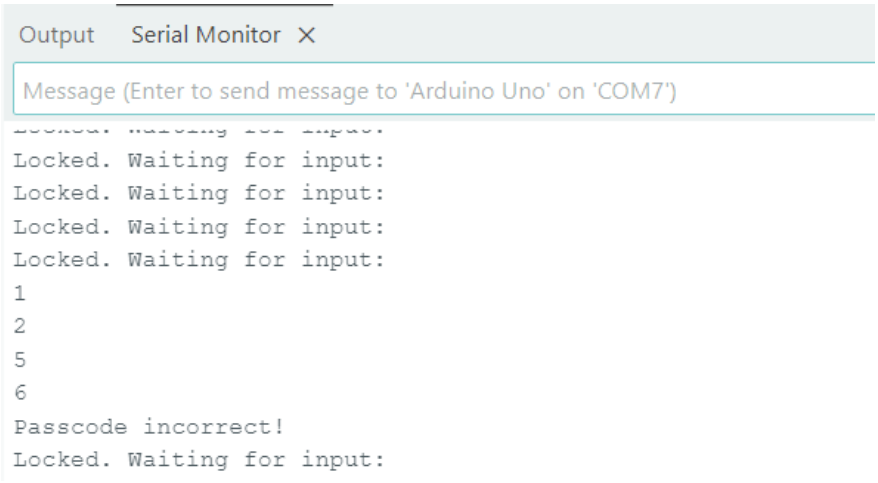

Figure 12: Serial Monitor Output in FSM Implementation on Incorrect Password

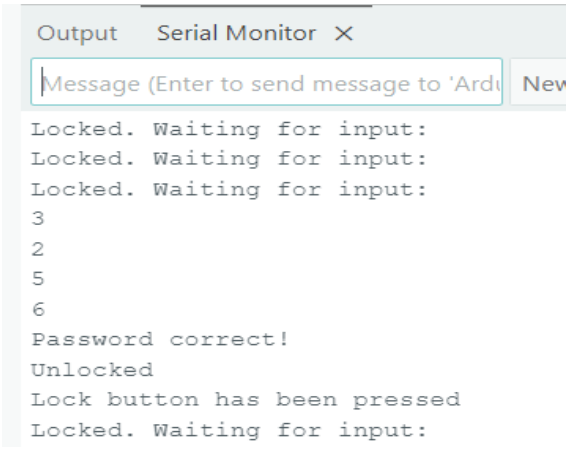

Figure 13: Serial Monitor Output Showcasing State Change from UNLOCKED to LOCKED on Boolean flag

```
void changePassword(char* password) {
  for (unsigned int i = 0; i < 4; i++) {
    EEPROM_write(i, password[i]);
  }
  EEPROM_write(4, '\0');
  EEPROM_write(PASSWORD_FLAG_ADDRESS, PASSWORD_FLAG);
}

void readPassword(char* input){
  for (unsigned int i=0;i<4;i++){
    input[i]= EEPROM_read(i);
  }
  input[4] = '\0';
}
```

Figure 14: Arduino Code utilizing EEPROM For changePassword() And readPassword()

Figure 15: Serial Monitor Output For Testing A Password Saved to EEPROM

```
bool motorBlocked(){
  int servoThreshold=75;
  int servoGPIO=A3;
  delay(800);
  int servoADC = analogRead(servoGPIO);
  if(servoADC>servoThreshold){
    return true;
  }
  return false;
}
```

Figure 16: motorBlocked() Helper Function

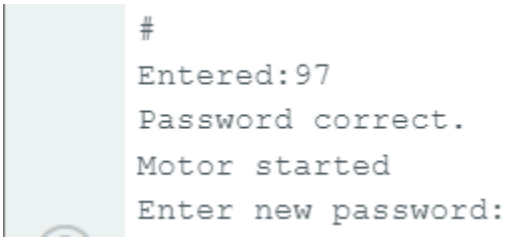

Figure 17: Serial Monitor Output Showcasing Non-Blocking Behavior Test for Servo Motor

```
Password correct.
Motor started
Servo blocked.
Motor stopped
```

Figure 18: Serial Monitor Output Showcasing Blocking Behavior Test for Servo Motor

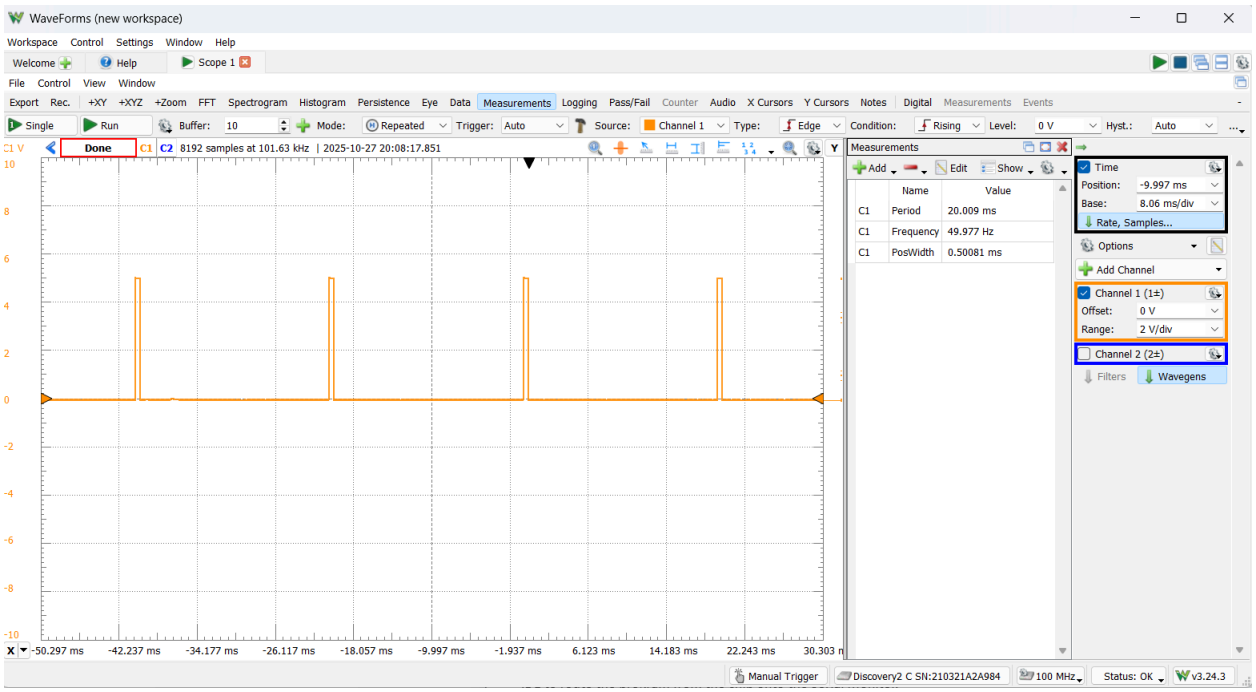

Figure 19: PWM Output Signal Measurement on Blocked Servo Motor

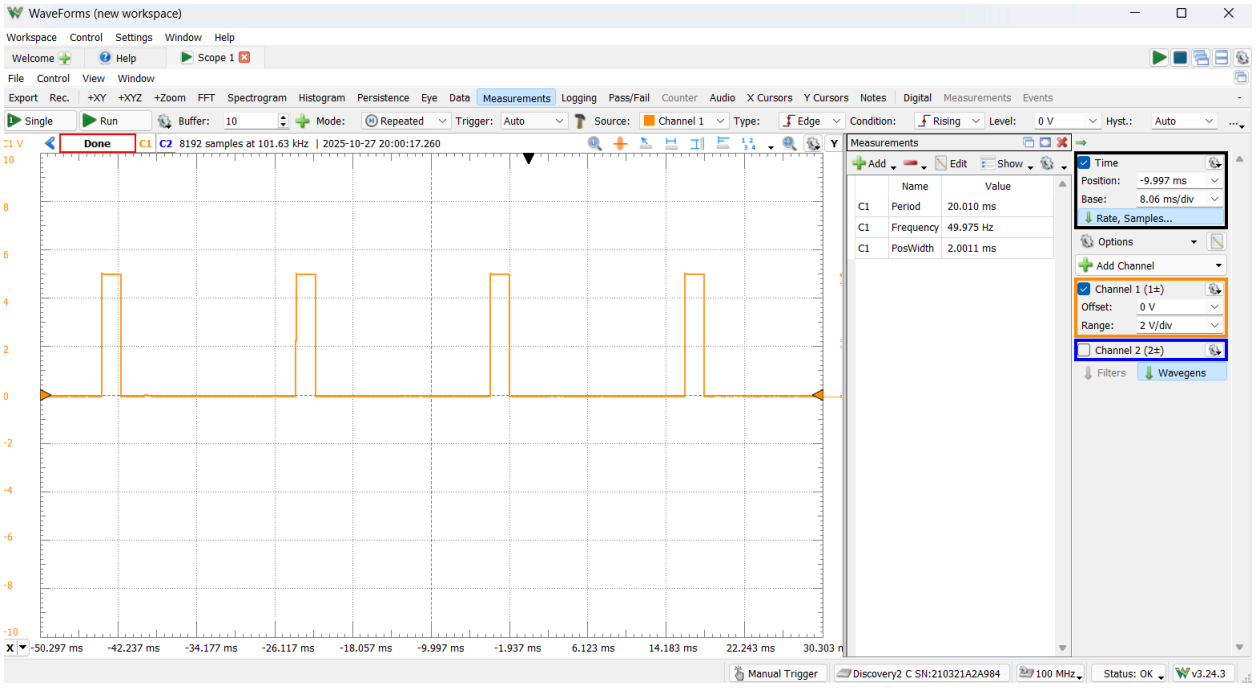

Figure 20: PWM Output Signal Measurement on Non-Blocked Servo Motor

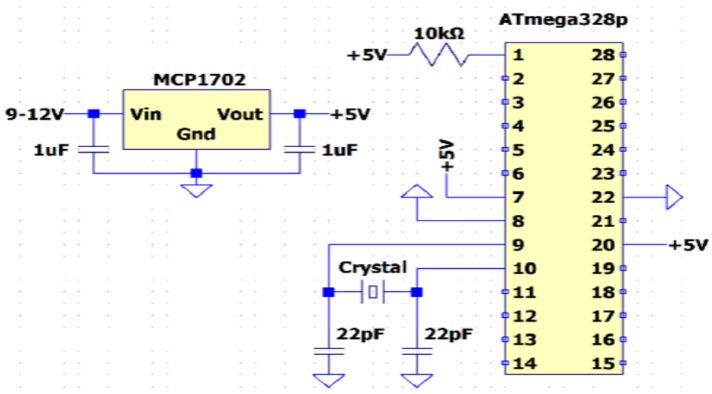

Figure 21: Circuit Diagram of Standalone ATmega328P and MCP1702 Regulator

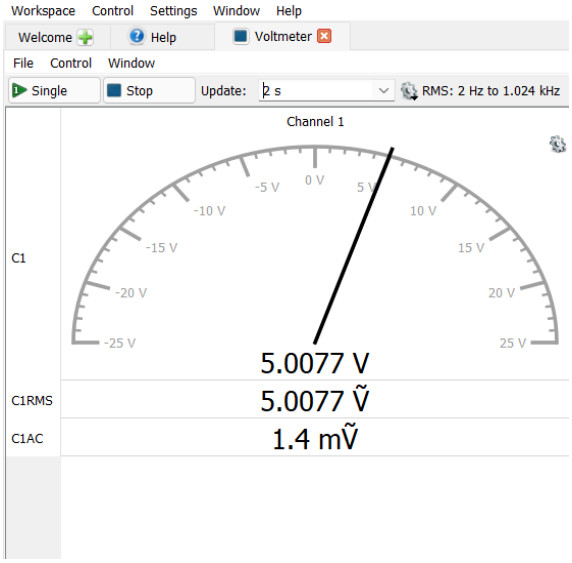

Figure 22: AD2 Voltmeter Reading across Output Node of MCP1702 Regulator

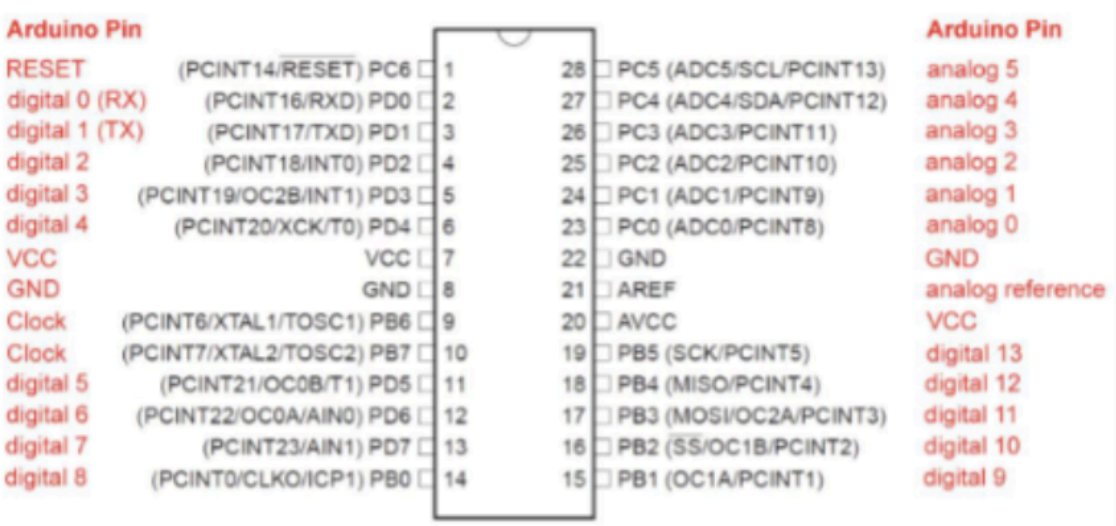

Figure 23: Diagram Mapping ATmega328P Pins to Arduino Uno Pins

	Column 1	Column 2	Column 3
Row 1	1 -> 1.62 V	2 -> 0.48 V	3 -> 2.51 V
Row 2	4 -> 4.14 V	5 -> 2.56 V	6 -> 4.55 V
Row 3	7 -> 3.41 V	8 -> 1.59 V	9 -> 4.11 V
Row 4	* -> 2.50 V	0 -> 0.89 V	# -> 3.40 V

Figure 24: Measured Voltages Across each Pressed Key on Keypad

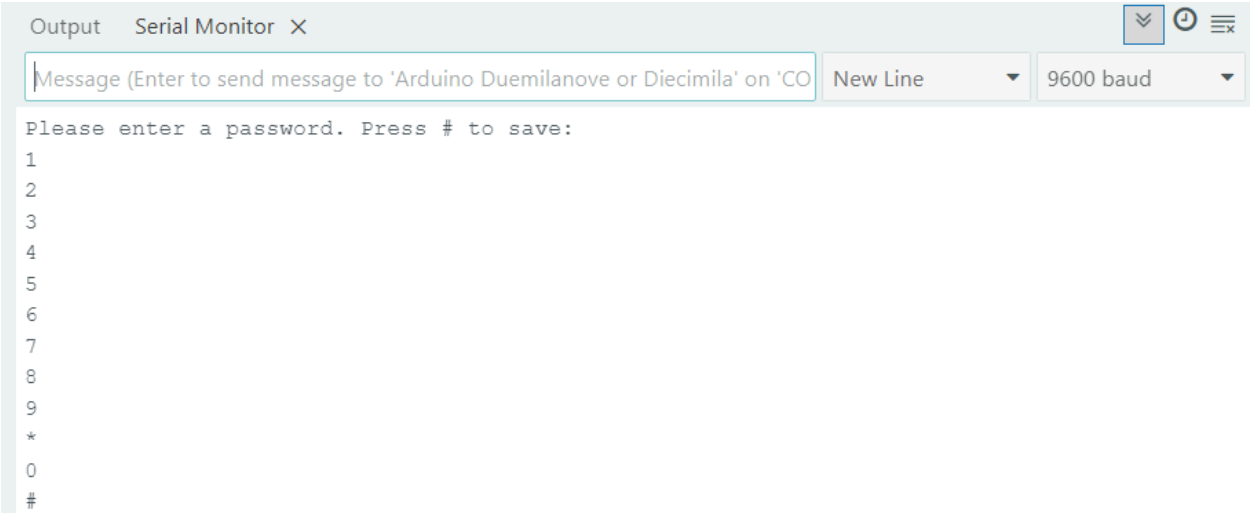

Figure 25: Serial Monitor Output From Locker Program Ran on Individual ATmega328P Testing Key Presses

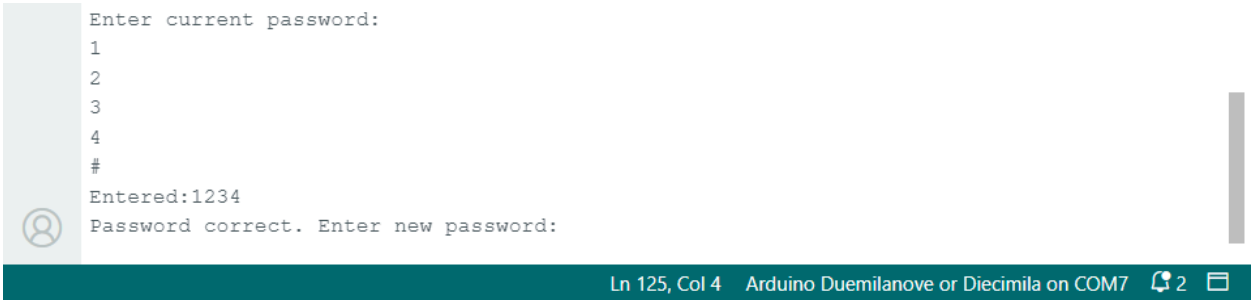

Figure 26: Serial Monitor Output on Standalone ATmega328P on Correct Password Behavior

```
2
2
8
#
New password saved.
#
Enter current password:
1
2
9
#
Entered:129
Incorrect password.
```

Ln 125, Col 4 Arduino Duemilanove or Diecimila on COM7 2

Figure 27: Serial Monitor Output on Standalone ATmega328p on Incorrect Password Behavior

```
5
7  void enterSleep() {
8    Serial.println("Going to sleep");
9    Serial.flush();
0    SMCR = (1<<SM1);  //set_sleep_mode(SLEEP_MODE_PWR_DOWN); //EXTERNAL
1    SMCR |= (1<<SE); // sleep_enable(); //external
2    __asm__ __volatile__("sleep" ::: "memory"); //sleep_cpu(); //external
3    SMCR &= ~(1<<SE); //sleep_disable(); //external
4    Serial.println("Woke up");
5  }
```

Figure 28: enterSleep() Function

```
ISR(PCINT1_vect) {
  keyInterrupt = true;
}
```

Figure 29: ISR Function For Keypad

```
void startMotorTimer() {
  cli();
  TCCR2A = 0; //resets registers and counter for timer2
  TCCR2B = 0;
  TCNT2  = 0;
  OCR2A = 31; //sets compare to 2ms
  TCCR2A |= (1 << WGM21); // CTC mode
  TCCR2B |= (1 << CS22) | (1 << CS21) | (1 << CS20); // prescaler 1024
  TIMSK2 |= (1 << OCIE2A); //enables interrupt
  sei();
}
```

Figure 30: startMotorTimer() Function

```
ISR(TIMER2_COMPA_vect) {
  static int subCount = 0; // counts 2ms ticks
  subCount++;
  if (subCount >= 250) { //250*2ms = 500ms
    subCount = 0;
    motorTimerCount++;
    if (motorTimerCount >= 10) { //10*500ms = 5s
      motorTimerExpired = true;
      motorRunning = false;
      TIMSK2 &= ~(1 << OCIE2A); //disable Timer2 interrupt
    }
  }
}
```

Figure 31: ISR Function For Motor Timer

```
if (key == NULL) {
  if (!keyInterrupt && !motorRunning) {
    enterSleep();
  } else {
    keyInterrupt = false; //clear interrupt flag
  }
  return;
}
```

Figure 32: Main Loop Logic Using Interrupt Flags and Sleep

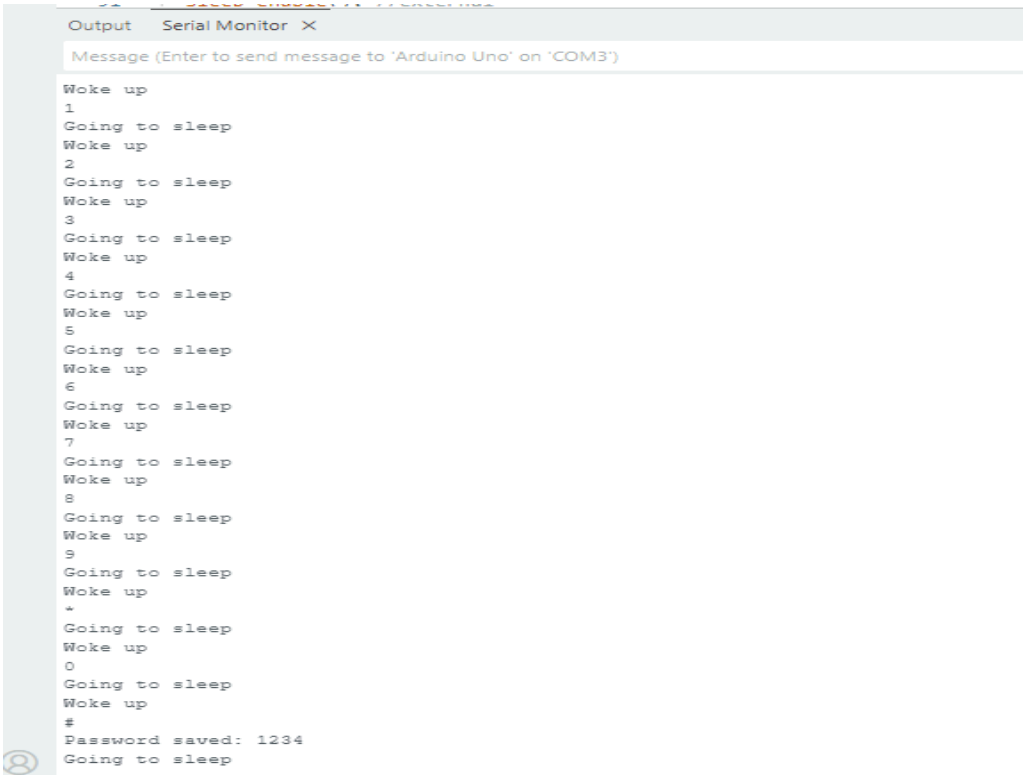

Figure 33: Correct Keypad Behavior Incorporating Interrupts And Power-Down Mode

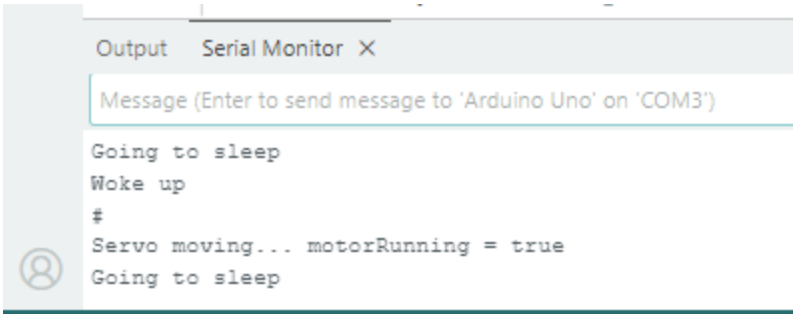

Figure 34: Correct Motor Running Behavior Incorporating Timer Interrupts and Power Down Mode

```
bool lowBattery(){
  //6.5V is threshold for 9V battery, becomes 3.25V with voltage divider, (3.25/5)1023 ~= 670 for threshold
  int lowBatteryThreshold =665;
  int lowBatteryGPIO = A5;
  int lowBatteryADC = analogRead(lowBatteryGPIO);

  if(lowBatteryADC<lowBatteryThreshold){
//    lowBattery = true;
    return true;
  }

//  lowBattery=false;
  return false;
}
```

Figure 35: lowBattery() Function

```
if (key == NULL) {
  if (!keyInterrupt && !motorRunning &!lowBattery()) {
    enterSleep();
  } else {
    keyInterrupt = false; //clear interrupt flag
  }
  return;
}
```

Figure 36: Modified Conditional for Sleep Mode

```
if (lowBattery()==true){
  digitalWrite(ledPin, HIGH);
  delay(1000);
  digitalWrite(ledPin, LOW);
  delay(1000);
}
```

Figure 37: Using lowBattery() To Control LED Blinking.

https://drive.google.com/file/d/1ufPoe34asEIYhXf1fEIrGV9mWAkg2IuG/view?usp=sharing

Figure 38: LED Off With 9V Source

https://drive.google.com/file/d/1L11NuY-eRbByc0aD_HBEQBIuVlaDMPso/view?usp=sharing

Figure 39: Blinking LED With 5V Source

```
372  void playTone(int pin, int frequency, int duration) {
373    int period = 1000000 / frequency;
374    int pulse = period / 2;
375    unsigned long endTime = millis() + duration;
376    while (millis() < endTime) {
377      digitalWrite(pin, HIGH);
378      delayMicroseconds(pulse);
379      digitalWrite(pin, LOW);
380      delayMicroseconds(pulse);
381    }
382  }
```

Figure 40: playTone Helper function to handle tone() Behavior

```
void incorrectPasswordAudioQueue(int speakerPin) {
  playTone(speakerPin, 440, 200);
  playTone(speakerPin, 340, 200);
  playTone(speakerPin, 240, 600);
}
void correctPasswordAudioQueue(int speakerPin) {
  playTone(speakerPin, 650, 200);
  playTone(speakerPin, 800, 200);
  playTone(speakerPin, 950, 600);
}
void relockAudioQueue(int speakerPin) {
  playTone(speakerPin, 220, 200);
}
```

Figure 41: Audio Queue Functions for Each Different State of Locker

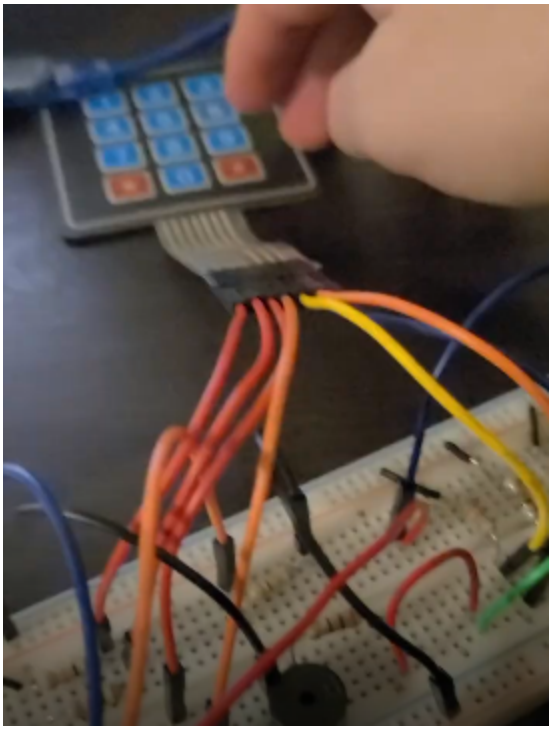

Figure 42: Correct Password Audio Queue Test File Image

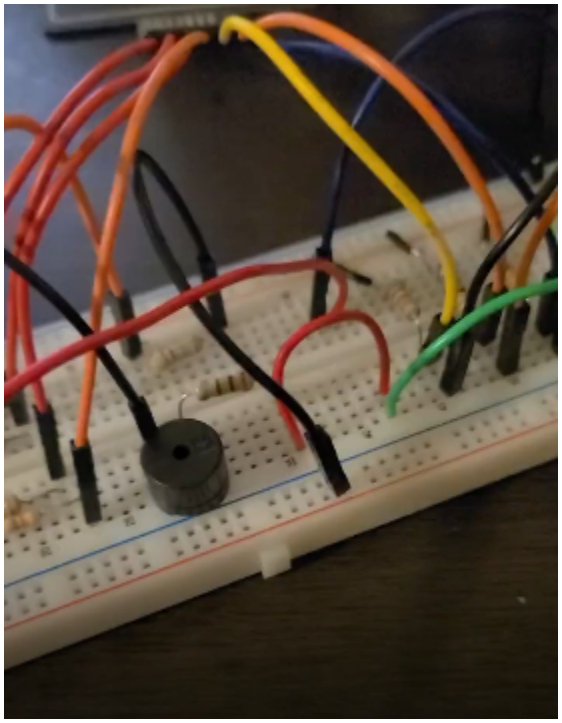

Figure 43: Incorrect Password Audio Queue Test File Image

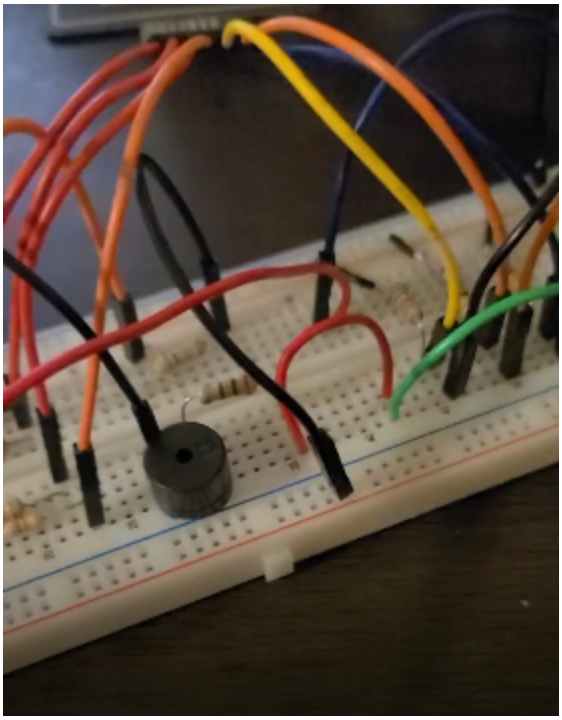

Figure 44: Relock Audio Queue Test File Image

```
void changePassword(char* password) {
  Serial.print("Encrypted Password:");
  for (unsigned int i=0; i<4; i++) {
    char encrypted = password[i] ^ XOR_KEY;
    Serial.print(encrypted);
    EEPROM_write(i, encrypted);
  }
  Serial.println(" ");
  EEPROM_write(4, '\0' ^ XOR_KEY);
  EEPROM_write(PASSWORD_FLAG_ADDRESS, PASSWORD_FLAG);
}
```

Figure 45: New changePassword() Function With Encryption and Printed Encryption

```
void readPassword(char* input) {
  for (unsigned int i=0; i<4; i++) {
    input[i] = EEPROM_read(i) ^ XOR_KEY;
  }
  input[4] = '\0';
}
```

Figure 46: New readPassword() Function With Decryption

```
Going to sleep
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ Please enter a password. Press # to save:
Going to sleep
Woke up
1
Going to sleep
Woke up
2
Going to sleep
Woke up
4
Going to sleep
Woke up
5
Going to sleep
Woke up
#
Encrypted Password:khno
Password saved: 1245
Going to sleep
Woke up
#
Enter current password:
Going to sleep
Woke up
1
Going to sleep
Woke up
2
Going to sleep
Woke up
4
Going to sleep
Woke up
5
Going to sleep
Woke up
#
Entered:1245
Password correct. Enter new password:
Going to sleep
```

Figure 47: Validated Output For EEPROM Password Encryption

```
bool lowBattery(){
  //6.5V is threshold for 9V battery, becomes 3.25V with voltage divider, (3.25/5)1023 ~= 670 for threshold
  int lowBatteryThreshold =665;
  int lowBatteryGPIO = A5;

  digitalWrite(nmosPin, HIGH);
  delayMicroseconds(10);
  int lowBatteryADC = analogRead(lowBatteryGPIO);
  digitalWrite(nmosPin, LOW);

  if(lowBatteryADC<lowBatteryThreshold){
        return true;
  }

//  lowBattery=false;
  return false;
}
```

Figure 48: Updated lowBattery() Function With 2N7000 Switch

https://drive.google.com/file/d/19uGrRCsh8_JFTMDesanFayZc2PvaJ_Vu/view?usp=drive_link

Figure 49: Functionality of Entire System

```
#define PASSWORD_FLAG_ADDRESS 10
#define PASSWORD_FLAG 0xA5
```

```
#define XOR_KEY 0x5A //random key for encryption


int col1 = A0;
int col2 = A1;
int col3 = A2;
int numKeys = 12;
char currentPassword[5];
char enteredPassword[5];
char newPassword[5];
int index;
bool passwordSet = false;
bool firstLoop=false;
//bool lowBattery =false;
const int ledPin = 13;
const int nmosPin =12;
const int speakerPin = 11;

int servoPin = 9;
const int servoMinUs = 1000;
const int servoMaxUs = 2000;

inline uint16_t servoUsToTicks(int us) {
  return us * 2;
}


volatile bool motorRunning = false; //for interrupts
volatile bool motorTimerExpired = false;
volatile bool keyInterrupt = false;
volatile int motorTimerCount = 0;


unsigned long previousBlinkMillis = 0;
const long blinkInterval = 1000;
bool ledState = false;


struct keySetup{
  char* key;
```

```
  int min;
  int max;
  int col;
};


keySetup keyMap[]{
  {"1", 225, 389, 1}, //col1
  {"4", 820, 1023, 1},
  {"7", 634, 798, 1},
  {"*", 430, 539, 1},
  {"2", 61, 112, 2},//col2
  {"5", 430, 593, 2},
  {"8", 265, 389, 2},
  {"0", 143, 225, 2},
  {"3", 439, 593, 3}, //col3
  {"6", 900, 1023, 3},
  {"9", 788, 870, 3},
  {"#", 634, 747, 3},
};


enum state{
  IDLE,
  SET_ORIGIONAL_PASSWORD,
  CHECK_PASSWORD,
  NEW_PASSWORD,
};


state currentState = IDLE;


unsigned char EEPROM_read(unsigned int uiAddress) //from datasheet
 {
 /* Wait for completion of previous write */
 while(EECR & (1<<EEPE))
 ;
 /* Set up address register */
 EEAR = uiAddress;
```

```
 /* Start eeprom read by writing EERE */
 EECR |= (1<<EERE);
 /* Return data from Data Register */
 return EEDR;
 }


void EEPROM_write(unsigned int uiAddress, unsigned char ucData){ //from
datasheet
 /* Wait for completion of previous write */
  while(EECR & (1<<EEPE));
 /* Set up address and Data Registers */
  EEAR = uiAddress;
  EEDR = ucData;
 /* Write logical one to EEMPE */
  EECR |= (1<<EEMPE);
 /* Start eeprom write by setting EEPE */
  EECR |= (1<<EEPE);
 }




bool passwordExists() {
  return EEPROM_read(PASSWORD_FLAG_ADDRESS) == PASSWORD_FLAG; //worked
with debugging
}


ISR(PCINT1_vect) {
  keyInterrupt = true; //interrupt for any key press
}


void enterSleep() {
  Serial.println("Going to sleep");
  Serial.flush();
  SMCR = (1<<SM1);  //set_sleep_mode(SLEEP_MODE_PWR_DOWN); //EXTERNAL
  SMCR |= (1<<SE); // sleep_enable(); //external
  __asm__ __volatile__("sleep" ::: "memory"); //sleep_cpu(); //external
```

```
  SMCR &= ~(1<<SE); //sleep_disable(); //external
  Serial.println("Woke up");
}


void startMotorTimer() {
  cli();
  TCCR2A = 0; //resets registers and counter for timer2
  TCCR2B = 0;
  TCNT2  = 0;
  OCR2A = 31; //sets compare to 2ms
  TCCR2A |= (1 << WGM21); // CTC mode
  TCCR2B |= (1 << CS22) | (1 << CS21) | (1 << CS20); // prescaler 1024
  TIMSK2 |= (1 << OCIE2A); //enables interrupt
  sei();
}


bool motorBlocked(){
  int servoThreshold=75;
  int servoGPIO=A3;
  delay(800);
  int servoADC = analogRead(servoGPIO);
  if(servoADC>servoThreshold){
    return true;
  }
  return false;
}


bool lowBattery(){
  //6.5V is threshold for 9V battery, becomes 3.25V with voltage divider,
(3.25/5)1023 ~= 670 for threshold
  int lowBatteryThreshold =665;
  int lowBatteryGPIO = A5;


  digitalWrite(nmosPin, HIGH);
  delayMicroseconds(10);
  int lowBatteryADC = analogRead(lowBatteryGPIO);
```

```
  digitalWrite(nmosPin, LOW);


  if(lowBatteryADC<lowBatteryThreshold){
        return true;
  }


//  lowBattery=false;
  return false;
}


void startMotor() {
  motorRunning = true;
  motorTimerExpired = false;
  motorTimerCount = 0;
  startMotorTimer();
  Serial.println("Motor started at ");
  Serial.println(millis());


  // servo unlocking code goes here
  servoUnlock();
}


void stopMotor() {
  motorRunning = false;
  Serial.println("Motor stopped at ");
  Serial.println(millis());
  // servo locking code goes here
  servoLock();
}


ISR(TIMER2_COMPA_vect) {
  static int subCount = 0; // counts 2ms ticks
  subCount++;
  if (subCount >= 250) { //250*2ms = 500ms
```

```
    subCount = 0;
    motorTimerCount++;
    if (motorTimerCount >= 10) { //10*500ms = 5s
      motorTimerExpired = true;
      motorRunning = false;
      TIMSK2 &= ~(1 << OCIE2A); //disable Timer2 interrupt
    }
  }
}

void incorrectPasswordAudioQueue(int speakerPin) {
  playTone(speakerPin, 440, 200);
  playTone(speakerPin, 340, 200);
  playTone(speakerPin, 240, 600);
}
void correctPasswordAudioQueue(int speakerPin) {
  playTone(speakerPin, 650, 200);
  playTone(speakerPin, 800, 200);
  playTone(speakerPin, 950, 600);
}
void relockAudioQueue(int speakerPin) {
  playTone(speakerPin, 220, 200);
}

void servoSetPulse(int us) {
  if (us < servoMinUs) us = servoMinUs;
  if (us > servoMaxUs) us = servoMaxUs;
  OCR1A = servoUsToTicks(us);
}

void servoUnlock() {
  servoSetPulse(servoMaxUs);
}

void servoLock() {
  servoSetPulse(servoMinUs);
}

void servoInit() {
  pinMode(servoPin, OUTPUT);
```

```
  TCCR1A = (1 << COM1A1) | (1 << WGM11);
  TCCR1B = (1 << WGM13) | (1 << WGM12) | (1 << CS11);

  ICR1 = 40000;
  OCR1A = servoUsToTicks(servoMinUs);
}


void setup(){
  Serial.begin(9600);
  Serial.flush();
  pinMode(ledPin, OUTPUT);
  pinMode(nmosPin, OUTPUT);
  pinMode(speakerPin, OUTPUT);
  digitalWrite(nmosPin, LOW);

  servoInit();


  PCICR |= (1 << PCIE1); // Enable Pin Change Interrupt
  PCMSK1 |= (1 << PCINT8) | (1 << PCINT9) | (1 << PCINT10);  // Enable A0,
A1, A2




  if(passwordExists()==true){
    passwordSet =true;
    readPassword(currentPassword);
  }
  else{
    passwordSet=false;
  }
}


void loop() {
if (lowBattery()) {
    unsigned long currentMillis = millis();
    if (currentMillis - previousBlinkMillis >= blinkInterval) {
```

```
        previousBlinkMillis = currentMillis;
        ledState = !ledState;
        if (ledState) {
          digitalWrite(ledPin, HIGH);
        } else {
          digitalWrite(ledPin, LOW);
        }
    }
} else {
    digitalWrite(ledPin, LOW); // Ensure LED is off if battery is not low
    ledState = false;
}
```

```
  if(firstLoop==false){
    Serial.println("Please enter a password. Press # to save:");
    currentState = SET_ORIGIONAL_PASSWORD;
    firstLoop=true;
  }

  const char* key = readKey();
    if (key == NULL) {
      if (!keyInterrupt && !motorRunning &!lowBattery()) {
        enterSleep();
      } else {
        keyInterrupt = false; //clear interrupt flag
      }
      return;
    }
```

```
  switch (currentState) {
    case IDLE:
      if (strcmp(key, "#") == 0) {
        index = 0;
```

```
    if (!passwordExists()) {
      currentState = SET_ORIGIONAL_PASSWORD;
      Serial.println("Enter new password, then press # to save:");
    } else {
      currentState = CHECK_PASSWORD;
      Serial.println("Enter current password:");
    }
  }
  break;


case SET_ORIGIONAL_PASSWORD:
  if (strcmp(key, "#") == 0) {
    newPassword[index] = '\0';
    changePassword(newPassword);
    strcpy(currentPassword, newPassword);
    passwordSet = true;
    Serial.print("Password saved: ");
    Serial.println(currentPassword);
    currentState = IDLE;
  } else if (index < 4) {
    newPassword[index++] = key[0];
  }
  break;


case CHECK_PASSWORD:
  if (strcmp(key, "#") == 0) {
    enteredPassword[index] = '\0';
    Serial.print("Entered:");
    Serial.println(enteredPassword);
    if (strcmp(enteredPassword, currentPassword) == 0) {
      Serial.println("Password correct. Enter new password:");
      correctPasswordAudioQueue(speakerPin);
      startMotor();
      index = 0;
      currentState = NEW_PASSWORD;
    } else {
      Serial.println("Incorrect password.");
      incorrectPasswordAudioQueue(speakerPin);
```

```
          currentState = IDLE;
        }
      } else if (index < 4) {
        enteredPassword[index++] = key[0];
      }
      break;


    case NEW_PASSWORD:
      if (strcmp(key, "#") == 0) {
        newPassword[index] = '\0';
        changePassword(newPassword);
        strcpy(currentPassword, newPassword);
        Serial.println("New password saved.");
        relockAudioQueue(speakerPin);
        currentState = IDLE;
      } else if (index < 4) { //check
        newPassword[index++] = key[0];
      }
      break;
  }


  if (motorTimerExpired) {
    stopMotor();
  motorTimerExpired = false;
  }


  delay(100);
}


void changePassword(char* password) {
  Serial.print("Encrypted Password:");
  for (unsigned int i=0; i<4; i++) {
    char encrypted = password[i] ^ XOR_KEY;
    Serial.print(encrypted);
    EEPROM_write(i, encrypted);
  }
```

```
  Serial.println(" ");
  EEPROM_write(4, '\0' ^ XOR_KEY);
  EEPROM_write(PASSWORD_FLAG_ADDRESS, PASSWORD_FLAG);
}


void readPassword(char* input) {
  for (unsigned int i=0; i<4; i++) {
    input[i] = EEPROM_read(i) ^ XOR_KEY;
  }
  input[4] = '\0';
}


const char* readKey(){
  int adc1 = analogRead(col1);
  int adc2 = analogRead(col2);
  int adc3 = analogRead(col3);


  for(int i=0; i<numKeys; i++){
    int adc = 0;


    if(keyMap[i].col == 1){
      adc = adc1;
    }
    else if(keyMap[i].col ==2){
      adc = adc2;
    }
    else if (keyMap[i].col ==3){
      adc = adc3;
    }


    if ((adc>=keyMap[i].min)&&(adc<=keyMap[i].max)){
      Serial.println(keyMap[i].key);
      delay(300);
      return keyMap[i].key;
    }
```

```
  }
  return NULL;
}

void playTone(int pin, int frequency, int duration) {
  int period = 1000000 / frequency;
  int pulse = period / 2;
  unsigned long endTime = millis() + duration;
  while (millis() < endTime) {
    digitalWrite(pin, HIGH);
    delayMicroseconds(pulse);
    digitalWrite(pin, LOW);
    delayMicroseconds(pulse);
  }
}
```

Figure 50: Overall Code for Locker Program